Information Retrieval, Retrieval Augmented Generation

Robin Jia & Xuezhe Ma
USC CSCI 544, Spring 2026
March 10, 2026

Logistics

- HW3 has been released, due Friday March 27

How do Language Models Know Things?

- *When was the Declaration of Independence signed?*
  - Easy question because this is frequently discussed in pre-training data
  - Model can directly predict the answer (*July 4, 1776*)
- What about…
  - Questions about recent events?

Model Internal Knowledge is Old!

ChatGPT
March 9, 2026
(yesterday)

What should I know about working for the Department of War?

2/2

It depends what you mean by "Department of War," since that name isn't currently used in the U.S.

In the United States, the **Department of War** existed from 1789 to 1947. It was replaced by the **Department of the Army**, and then folded into the **Department of Defense (DoD)** in 1947. So today, you'd be working for:

- **Department of Defense (DoD)** overall
- Or a branch like the **Army, Navy, Air Force, Marine Corps, Space Force**
- Or as a **civilian federal employee** within one of those organizations

Here's what you should know about working in today's equivalent:

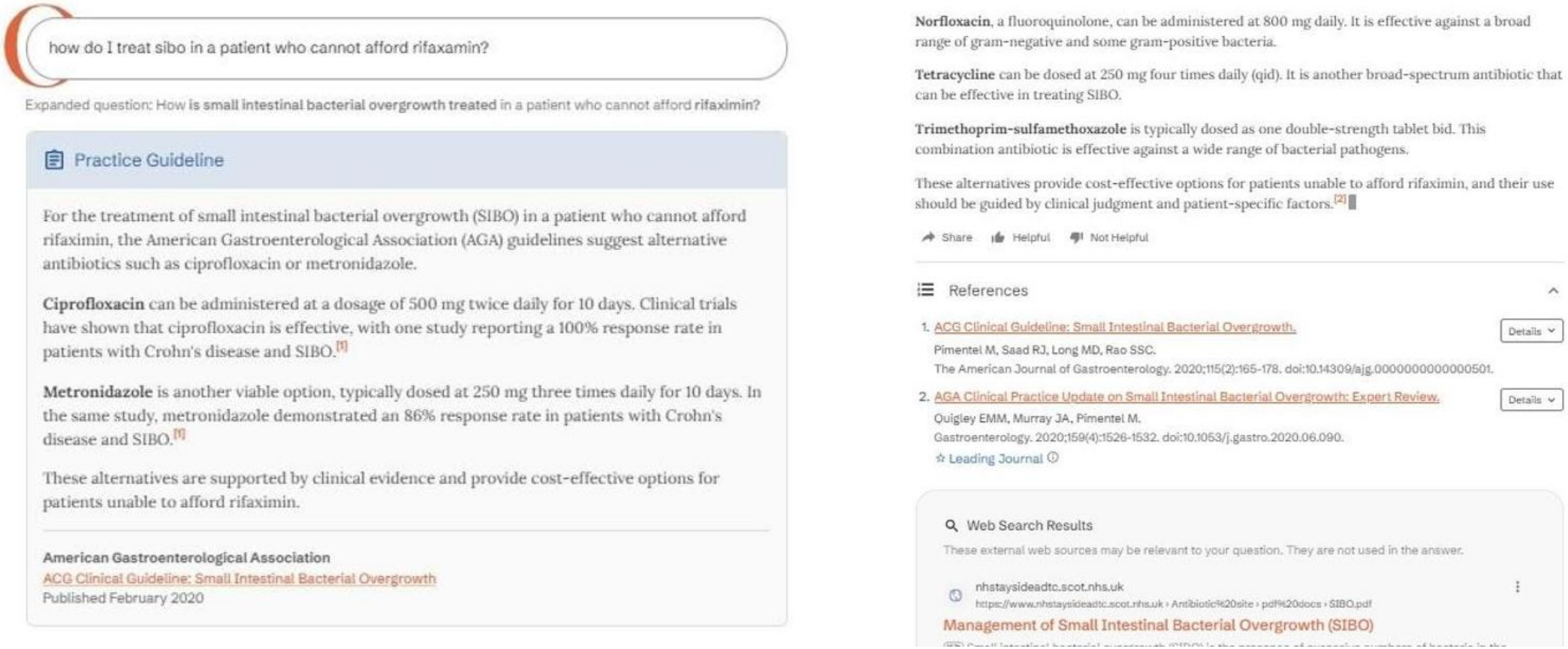
White House Website
September 5, 2025
(six months ago)

LM was trained many months ago; its internal knowledge is old! (Has a "knowledge cutoff date")

How do Language Models Know Things?

- *When was the Declaration of Independence signed?*
  - Easy question because this is frequently discussed in pre-training data
  - Model can directly predict the answer (*July 4, 1776*)
- What about…
  - Questions about recent events?
  - Questions about long-tail knowledge?

Supporting Long-Tail Knowledge Queries



OpenEvidence: LLMs provide answers & citations
by looking at relevant medical journal articles/websites

How do Language Models Know Things?

- *When was the Declaration of Independence signed?*
  - Easy question because this is frequently discussed in pre-training data
  - Model can directly predict the answer (*July 4, 1776*)
- What about…
  - Questions about recent events?
  - Questions about long-tail knowledge?
  - Questions about internal documents?
  - Questions about very long documents?

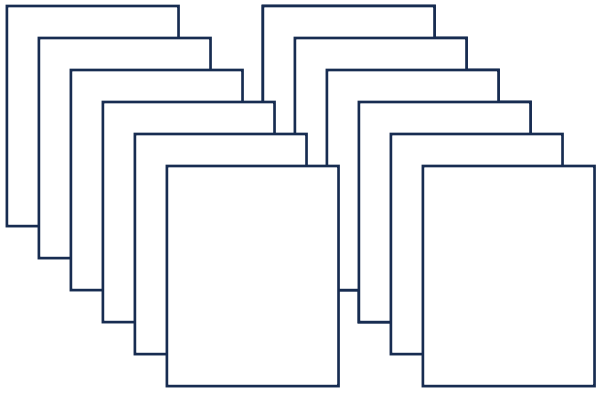
Outline

- Information Retrieval
  - Problem Definition
  - Sparse Retrieval
  - Dense Retrieval & Neural Models
- Retrieval Augmented Generation

Information Retrieval (IR)

- Input
  - A collection of documents (chosen ahead of time)
  - A text query (provided by user)
- Output
  - Ranked list of documents that best match the query

Document Collection

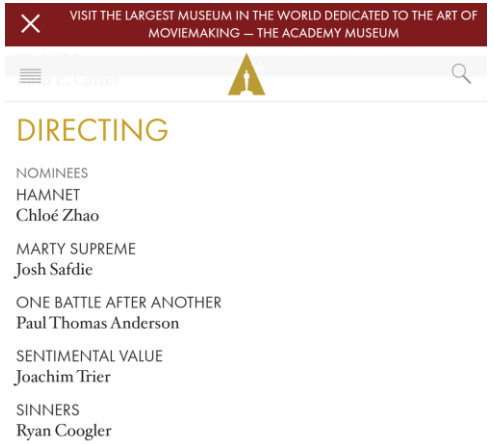


Query

Best Director Nominees

Ranked List:

#1



#2

2025 (98th)	Paul Thomas Anderson	*One Battle After Another*	[114]
	Ryan Coogler	*Sinners*	
	Josh Safdie	*Marty Supreme*	
	Joachim Trier	*Sentimental Value*	
	Chloé Zhao	*Hamnet*	

...

Two Key IR Challenges

Accuracy/Relevance

- Vast majority of documents are not relevant!
- Have to find the “needles in the haystack” that are relevant to the query

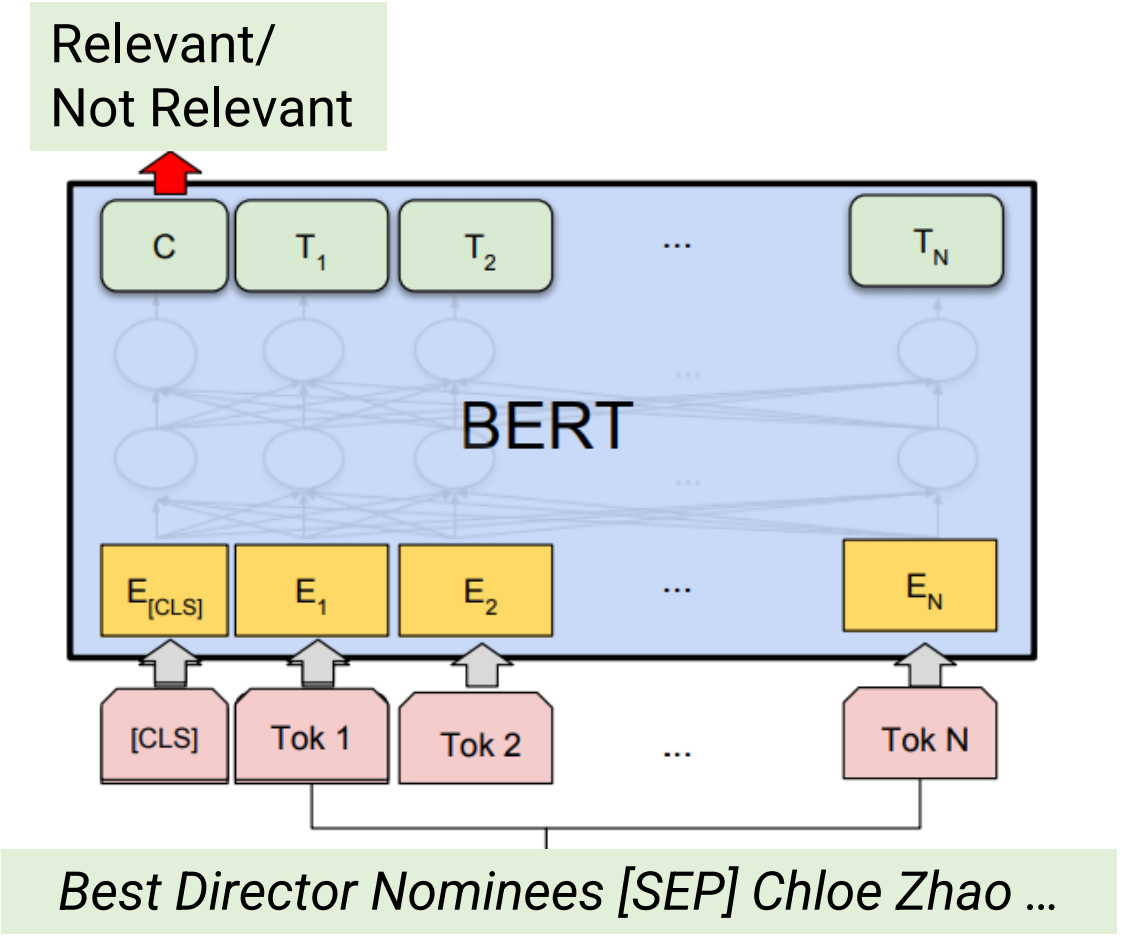
Latency

- PubMed lists 39 million biomedical papers
- Internet has ~3 billion webpages
- Google, etc. need to **quickly** locate the best pages out of those 3 billion!

https://commoncrawl.github.io/cc-crawl-statistics/plots/crawlsize

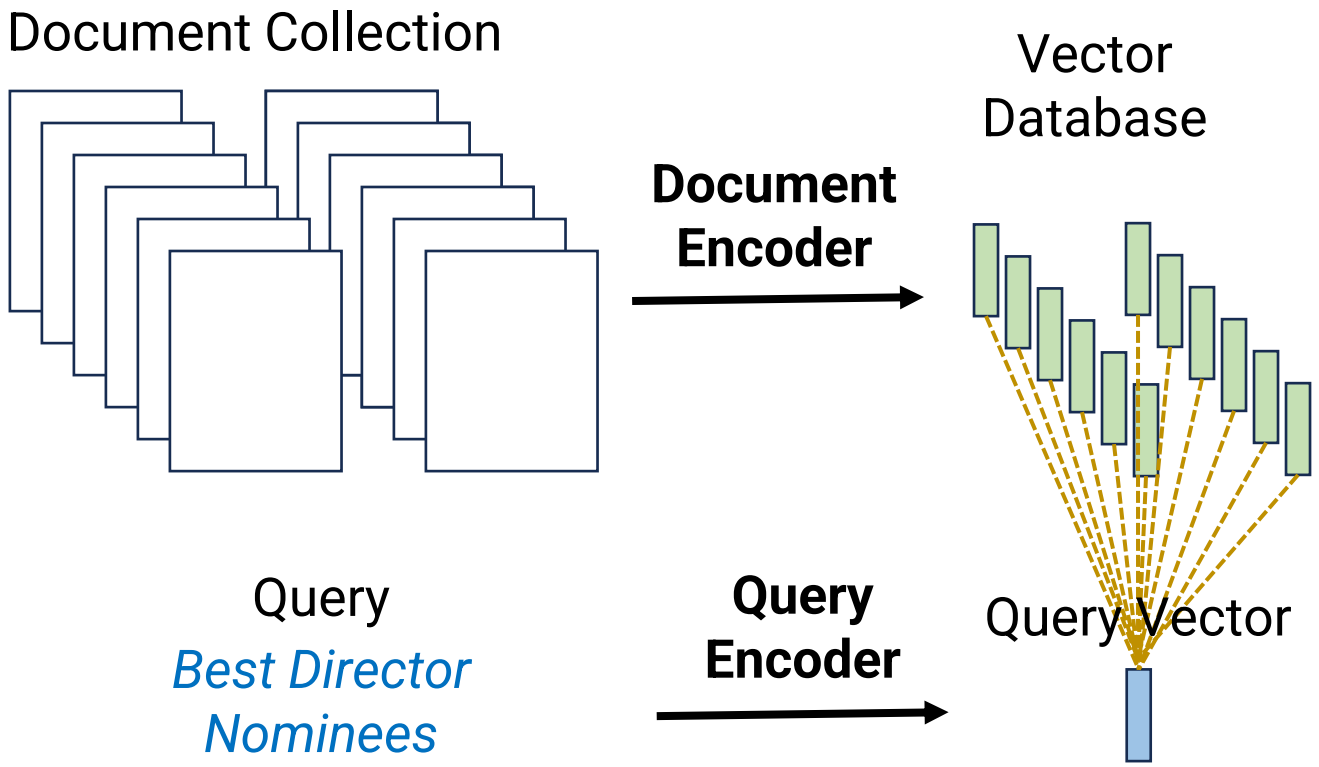
A Bad Approach to IR

- Train a relevance classifier
  - Input: (query, document)
  - Output: Probability that document is relevant to query
- Run classifier on every (query, document) pair
- Sort documents by probability of being relevant
- **Pro**: Can use sophisticated classifier to accurately predict relevance
- **Con**: This will be O(#documents) = Too slow!



Vector Space Model

- Key idea: **Vector Space Model**
  - Represent each document as a vector
  - Represent the query as a vector
  - Model relevance as vector similarity
- Why?
  - **Accuracy**: Can use different techniques to create good vector representations
  - **Latency**: Can identify most similar vectors efficiently with special algorithms

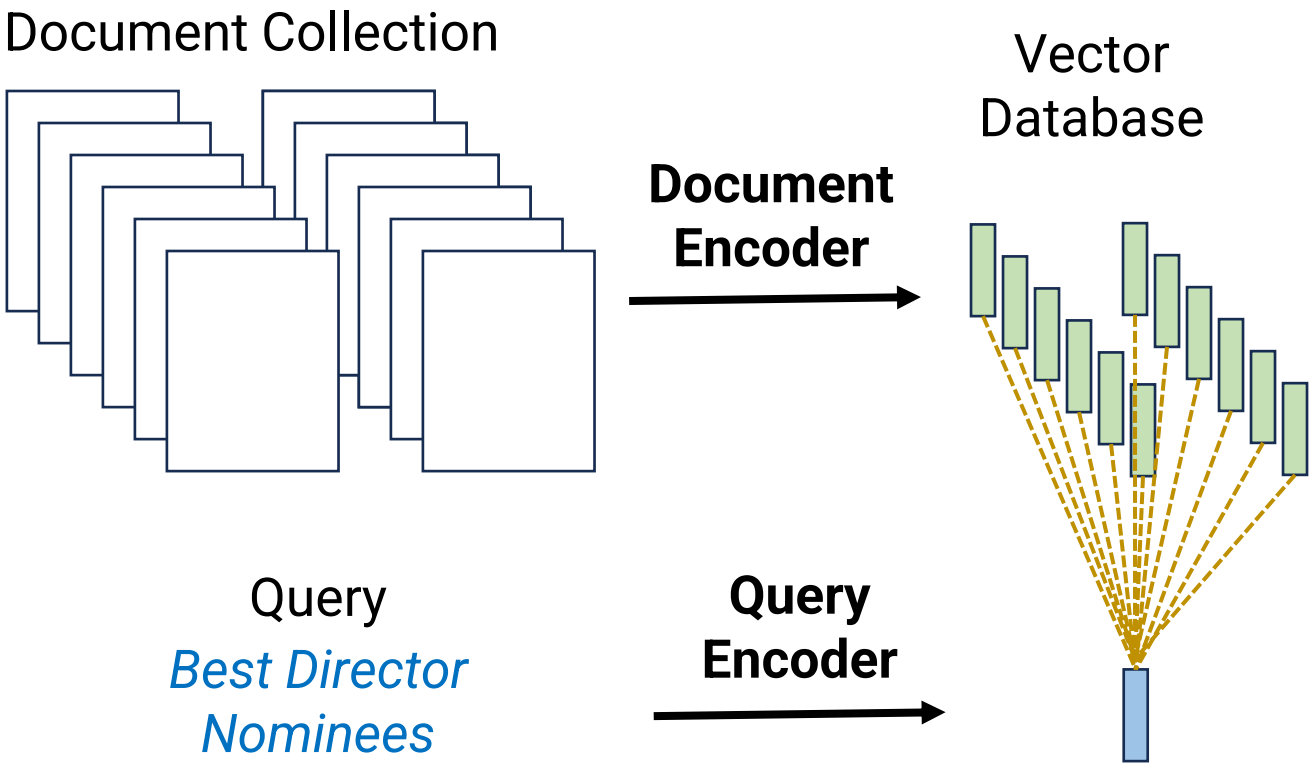


Outline

- Information Retrieval
  - Problem Definition
  - **Sparse Retrieval**
  - Dense Retrieval & Neural Models
- Retrieval Augmented Generation

Sparse Retrieval

- Traditional Approach: Represent queries and documents as **sparse** vectors
  - I.e., long vectors with many zero entries
- Measure similarity by cosine similarity between vectors $\frac{u^\top v}{\|u\|\|v\|} = \cos(\theta)$



A Bad Sparse Retrieval Approach

- Default: Let's use unigram features
  - Each document & query is mapped to |V|-sized vocabulary
  - i-th entry is the number of times the i-th vocabulary word appears

x = "great score and great movie"

Vocabulary *V*

Note: $d = |V| = 8$

index	word
1	acting
2	and
3	amazing
4	directing
5	great
6	movie
7	score
8	terrible

Features Φ(x)

index	value
1	0
2	1
3	0
4	0
5	2
6	1
7	1
8	0

A Bad Sparse Retrieval Approach

- Consider the following:
  - Query: *how should you care for orchids*
  - Document 1: *orchids require bright, indirect light*
  - Document 2: *you should care about sleep*
- With basic unigram features, Document 2 has (much) higher cosine similarity!
  - 3 matching words instead of 1
  - Same length = same normalization in cosine similarity
- What went wrong?
  - *"orchids"* is a word with important content
  - Other words are much less important => should carry less weight

Inverse Document Frequency

- In a collection of N documents, the **inverse document frequency** of a term *t* is:

$$\mathrm{idf}_t = \log_{10} \frac{N}{\mathrm{df}_t}$$

  - where df_t = number of documents where *t* occurs
- Intuition
  - If a word occurs in very few documents, it has high idf = important
  - If a word occurs in every document, it has idf = log(1) = 0
- Separately, we may entirely remove stopwords (*the*, *on*, etc.) that are too common to be useful

Term Frequency

- Consider the following:
  - Query: *should I grow tomatoes or peppers*
  - Document 1 (1000 words) mentions *tomatoes* 5 times and *peppers* 5 times
  - Document 2 (1000 words) mentions *tomatoes* 50 times and *peppers* 0 times
- Which is more relevant?
  - Intuitively, Document 1 is more likely to be relevant
  - But 10x "*tomatoes*" count would dominate if we're purely counting unigrams
- Solution: Instead of raw counts, define **term frequency** as

$$tf_{t,d} = \log_{10}(\#[\text{times term } t \text{ appears in document } d] + 1)$$

  - Logarithmic growth, so appearing 10x times does not yield 10x relevance
  - Note: $tf_{t,d} = \log(1) = 0$ when t does not appear in d
- BM25: Popular variant of tf-idf with more sophisticated tf definition

tf-idf vectors

- tf-idf vector for document d: i-th entry is $tf_{t,d}$ * idf_t
  - where t is the i-th vocabulary word
- idf term up-weights words that occur in fewer documents
- tf term incorporates term counts but attenuates with large counts

x = "great score and great movie"

Vocabulary *V*

Note: $d = |V| = 8$

index	word
1	acting
2	and
3	amazing
4	directing
5	great
6	movie
7	score
8	terrible

tf-idf vector

index	value
1	0
2	$\log(2) * idf_{and}$
3	0
4	0
5	$\log(3) * idf_{great}$
6	$\log(2) * idf_{movie}$
7	$\log(2) * idf_{score}$
8	0

tf-idf vector similarity

- Cosine similarity using tf-idf vectors:

tf-idf for query

tf-idf for document

$$\sum_{t \in q} \frac{\mathrm{tf}_{t,q} \cdot \mathrm{idf}_t}{\sqrt{\sum_{u \in q} \mathrm{tf}_{u,q}^2 \cdot \mathrm{idf}_u^2}} \cdot \frac{\mathrm{tf}_{t,d} \cdot \mathrm{idf}_t}{\sqrt{\sum_{u \in d} \mathrm{tf}_{u,d}^2 \cdot \mathrm{idf}_u^2}}$$

Normalization to unit vector from cosine similarity formula

Sum only over terms in query = efficient!

- Note: Sum only over query terms since $\mathrm{tf}_{t,q} = 0$ when t not in q

Efficient Querying with Sparse Vectors

- How do we retrieve the most similar documents?
  - **Not willing** to do O(#documents) computation per query
  - **Are willing** to do O(#documents) computation as pre-processing
- Idea: Build an **inverted index** data structure
  - Dictionary where each term (vocabulary word) is a key
  - Associated value is list of document ID's containing that term ("posting list")
- Example: {

  "tomatoes": [d_3, d_{23}, d_{80}, …], ← posting list

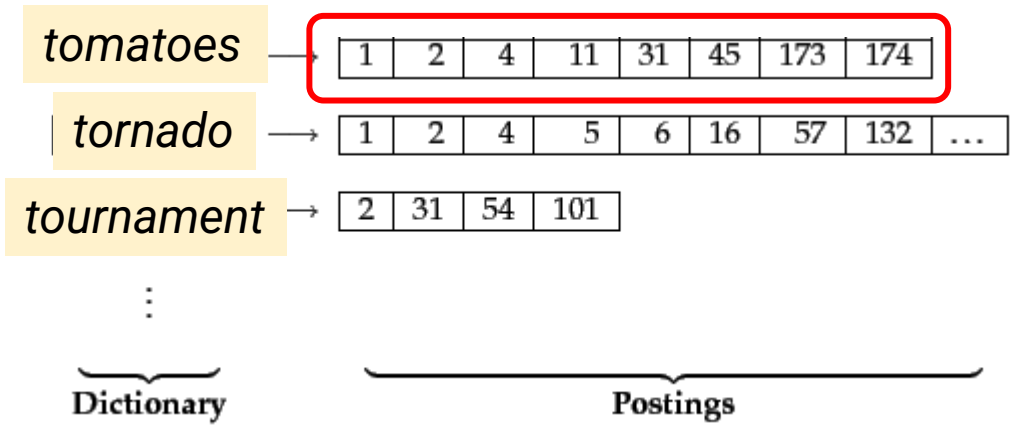
  "peppers": [d_{17}, d_{64}, d_{92}, …],

  …

}

Using the Inverted Index

- Query: *should I grow tomatoes or peppers*
- Basic idea: Only look at posting lists for terms in the query!
  - Any document that is in none of the posting lists has 0 common words with the query
  - Therefore it has 0 cosine similarity, can safely ignore it
  - We compute full tf-idf similarity for documents in the relevant posting lists



Using the Inverted Index

- Query: *should I grow tomatoes or peppers*
- Many optimizations possible to find top K documents only
  - Only look at posting lists for terms with high idf score
    - E.g., Only look at posting lists for *tomatoes* and *peppers*
    - Equivalent to, only consider documents that contain at least 1 of those words
    - Saves a lot of time because by definition, low idf words have the longest posting lists!
  - Only look at documents that appear on multiple posting lists
    - E.g., Only look at intersection of posting lists for *tomatoes* and *peppers*
    - Posting lists typically sorted by document ID, to enable efficient intersections
  - Only look at **Champion lists** = For each term *t*, store the R documents with highest weights for *t*
- Can fall back to more exhaustive search if these return < K documents

Query Expansion

- Consider the following:
  - Query: *should I grow tomatoes or peppers*
  - Document 1 mentions *tomatoes* 0 times and *peppers* 0 times, **but** mentions *tomato* 50 times and *pepper* 50 times
  - This is probably still a good match!
  - Naïve tf-idf would miss this since there's no exact word matching
- Query Expansion: Given a query, automatically add very similar words to the query
  - Use a thesaurus
  - Use domain knowledge (e.g., medical ontologies)
  - Use similar words (e.g., based on word vectors)

Sparse Retrieval Summary

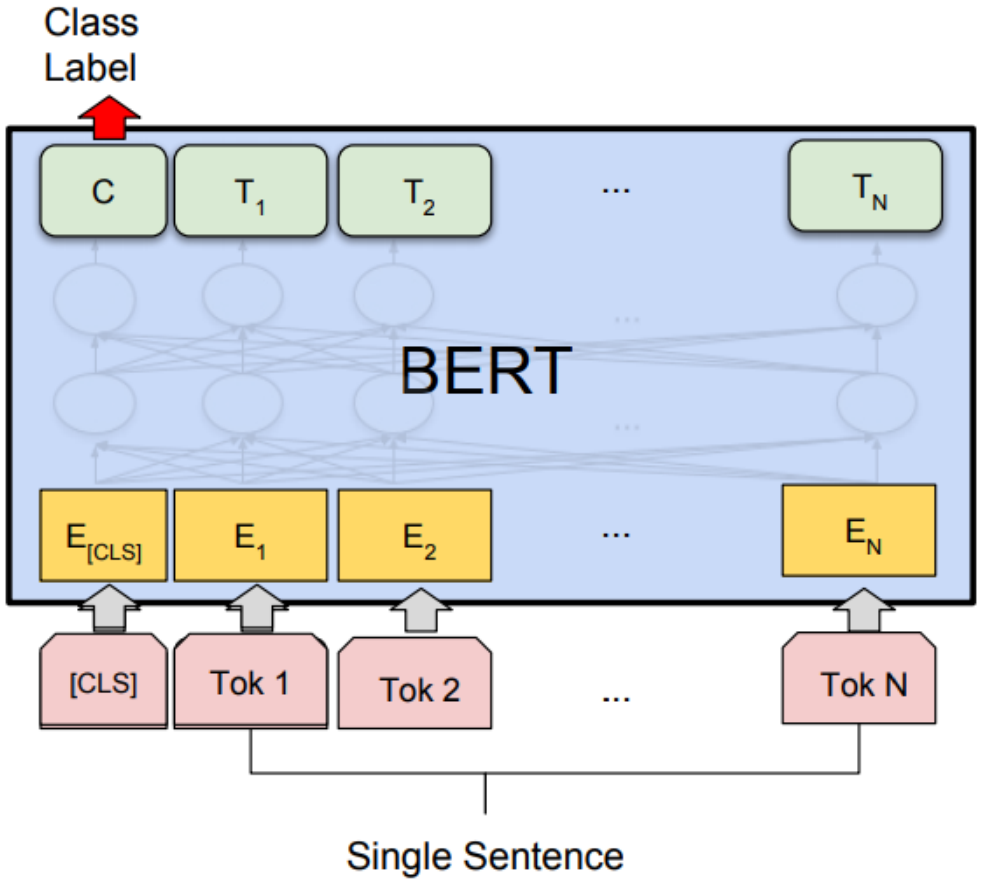
- **Good**: tf-idf or BM25 are surprisingly good at giving relevant documents
- **Good**: No “training” required besides counting words in document corpus
  - Note: BM25 does have a few parameters to tune
- **Great**: Inverted indices make it possible to be very efficient
- **Good**: Memory usage is not so bad
  - Term counts can be stored as sparse vectors with integer counts
  - idf scores are only stored per vocabulary word, doesn’t scale with number of documents
- **Bad**: Still a bag of words model, can easily miss relevant documents

Outline

- Information Retrieval
  - Problem Definition
  - Sparse Retrieval
  - **Dense Retrieval & Neural Models**
- Retrieval Augmented Generation

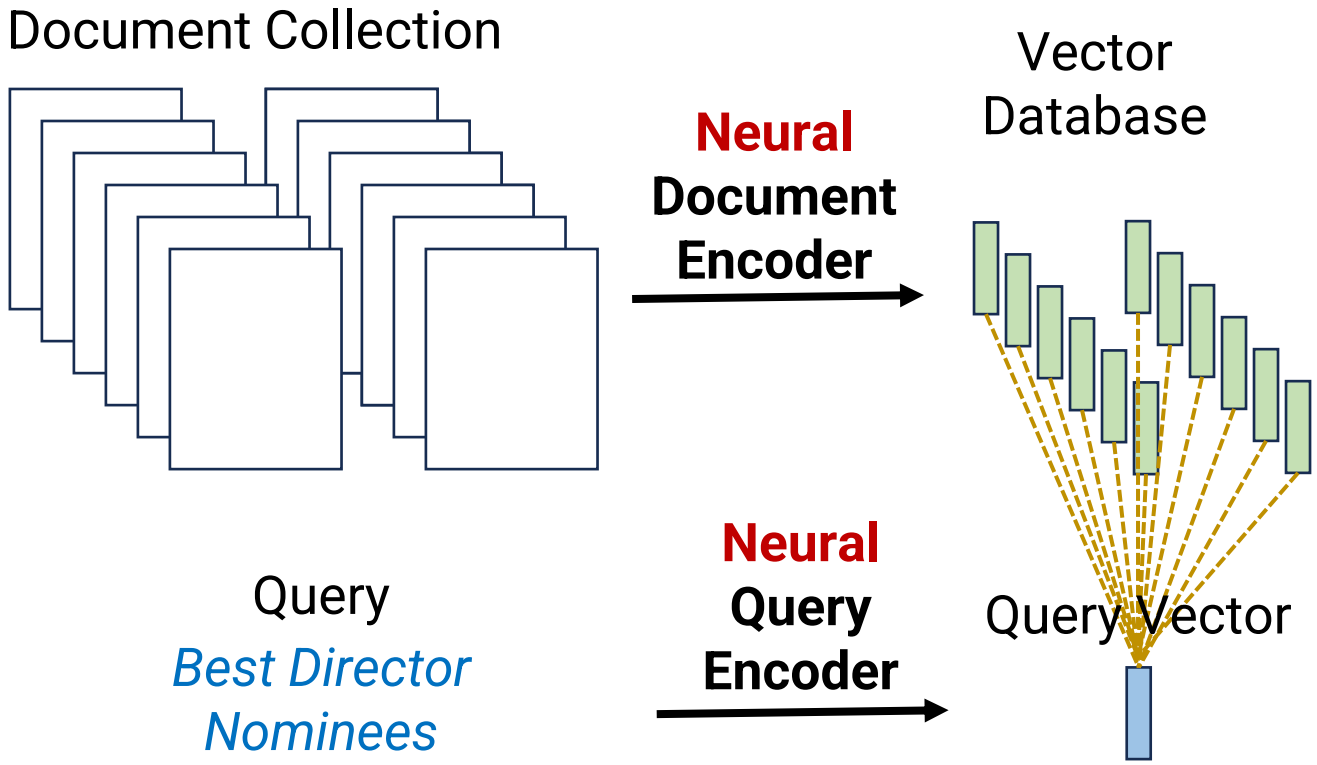
Dense Retrieval

- Can we leverage pre-trained neural models for retrieval?
- Two key questions:
  - **Accuracy**: How to train neural model to yield relevant results?
  - **Efficiency**: How to find relevant documents efficiently?



Single Vector Dense Retrieval

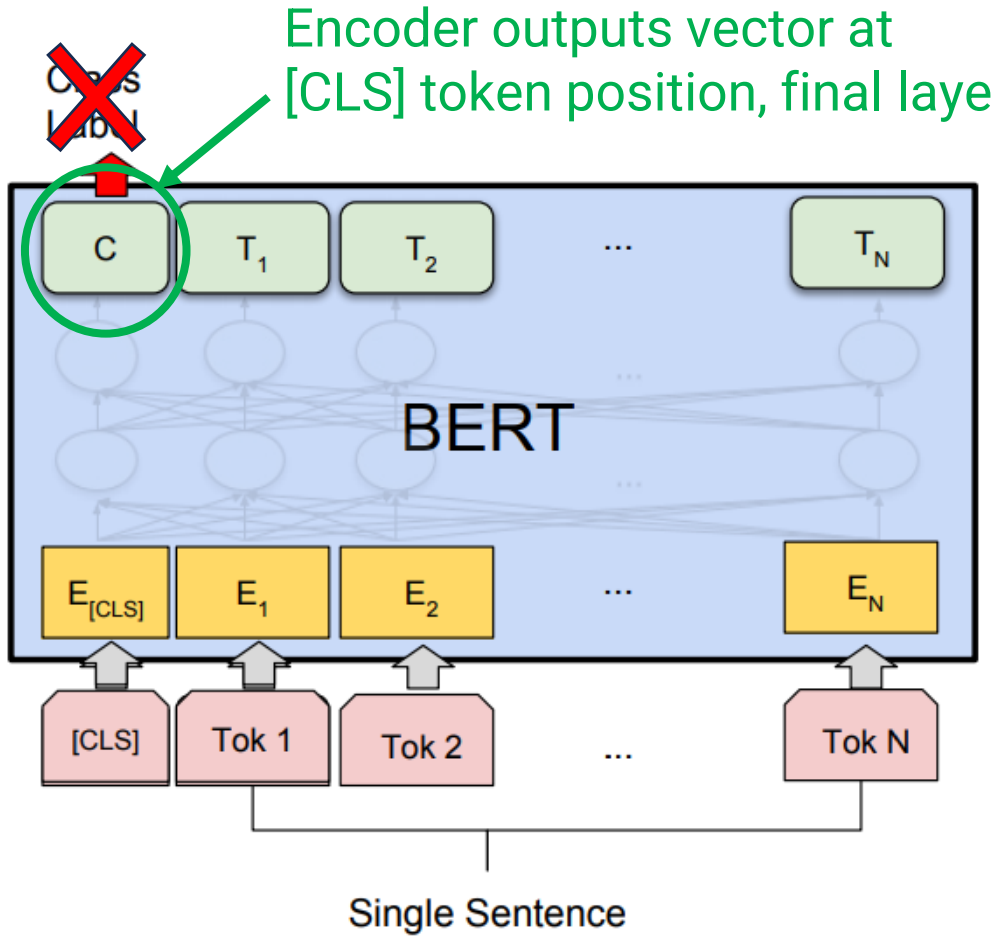
- First idea: Simply replace sparse tf-idf vectors with dense vectors derived from a neural model!
  - Each document & query gets mapped to 1 vector
  - Measure similarity with cosine similarity or dot product



Dense Passage Retrieval (DPR)

- DPR: Famous 2020 paper that made this work
- Model architecture: BERT
  - One fine-tuned BERT model E_Q to encode queries
  - Second fine-tuned BERT model E_P to encode passages
  - Similarity between query *q* and passage *p* is just dot product:

$$\text{sim}(p, q) = E_Q(q)^\top E_P(p)$$



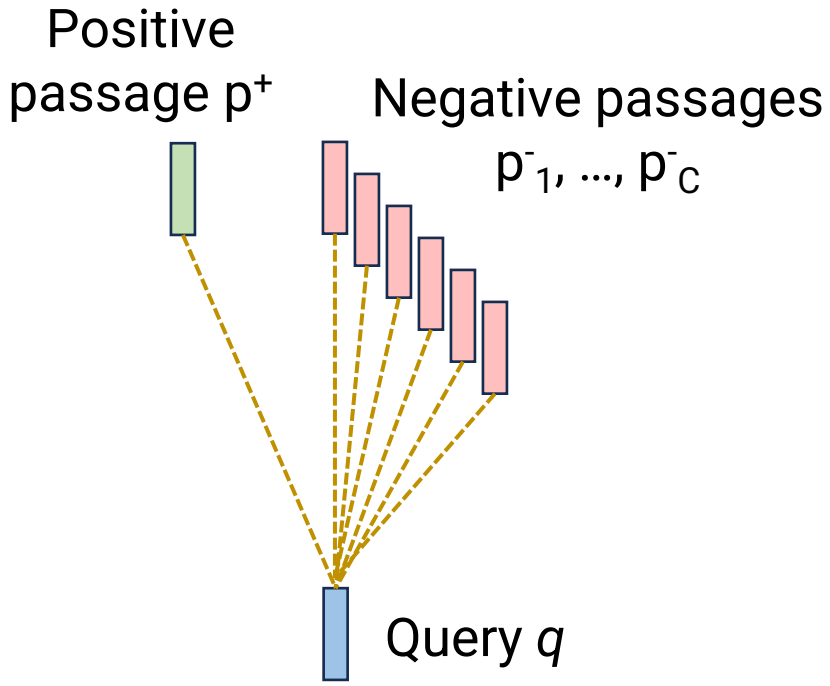
Training DPR

- Each training example consists of
  - Query q
  - Relevant (positive) passage p^+
  - C irrelevant (negative) passages p^-_1, …, p^-_C
- Training objective: Minimize

$$-\log \frac{e^{\mathrm{sim}(q,p^+)}}{e^{\mathrm{sim}(q,p^+)} + \sum_{j=1}^{C} e^{\mathrm{sim}(q,p_j^-)}}$$

Always minimize
-log P(correct answer)

Softmax function!
≈ K+1 way classification,
p^+ is the correct answer

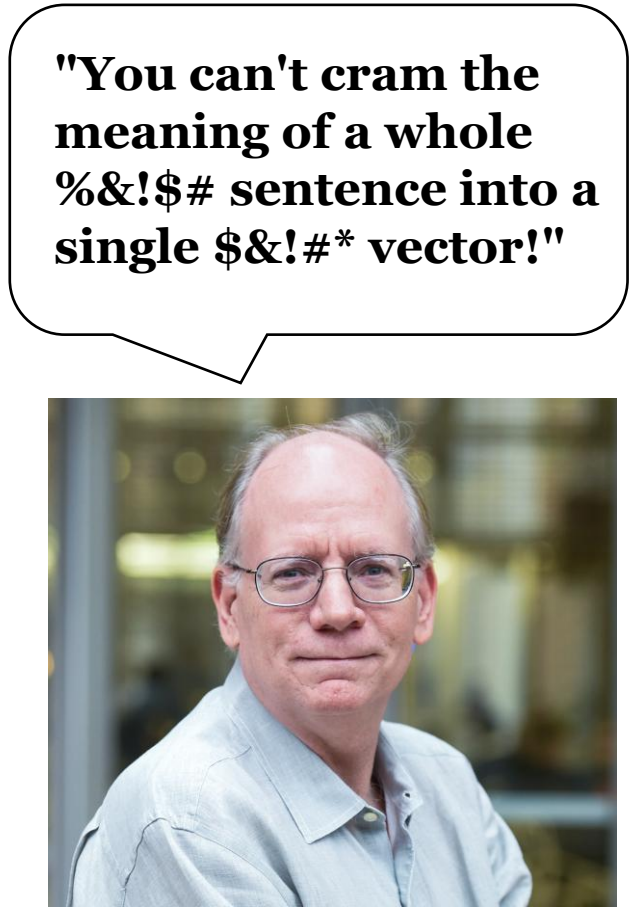


Creating Data for DPR

- Start with a question answering dataset
  - E.g., Natural Questions
  - Format is (question, answer) pairs
- Positive examples
  - Use highest-ranked passage retrieved by BM25 that contains answer
- Negative examples
  - Hard negatives: Passages retrieved by BM25 that **do not** contain answer
    - Advantage: Trains encoder to **recognize subtle distinctions**
  - Random in-batch negatives: When training with mini-batch, use other (positive & negative) passages in batch as additional negatives
    - Advantage: Already encoding these passages, so it's **"free" training data**

Document Chunking

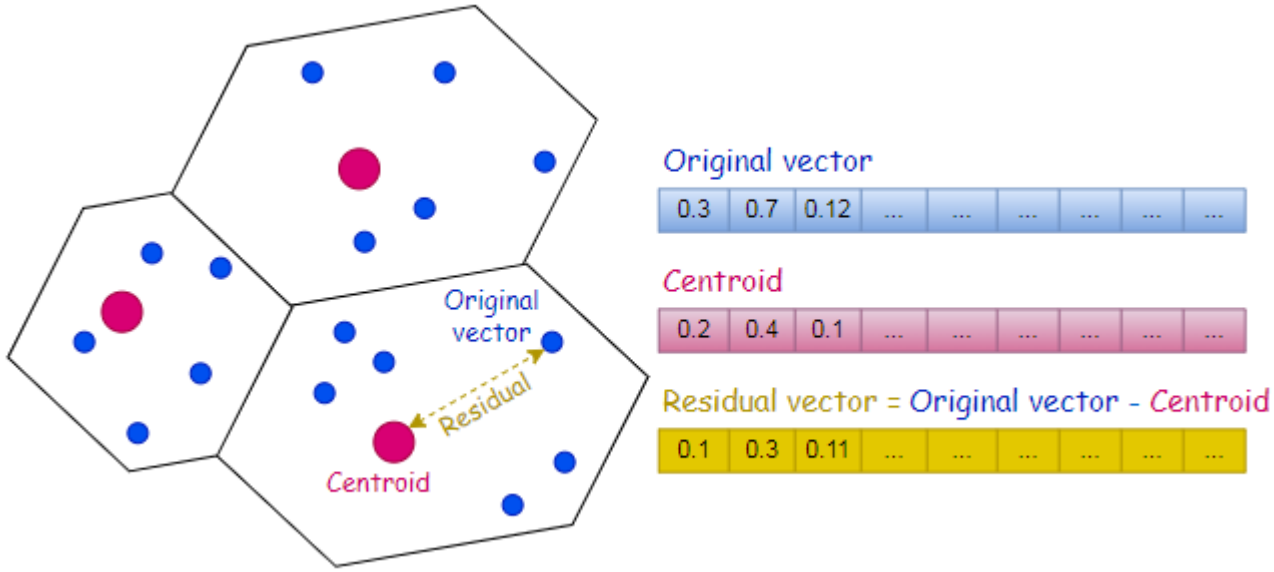
- Important detail: Many documents are very long!
  - May be longer than context length of the model
  - Even if model can process it, can a single vector summarize everything about the document?
- Solution: Document chunking
  - Break documents into many smaller chunks
  - DPR: Split Wikipedia articles into disjoint text chunks of 100 words, prepend title of article to each chunk
    - Resulted in ~21 million chunks of Wikipedia



Prof. Ray Mooney

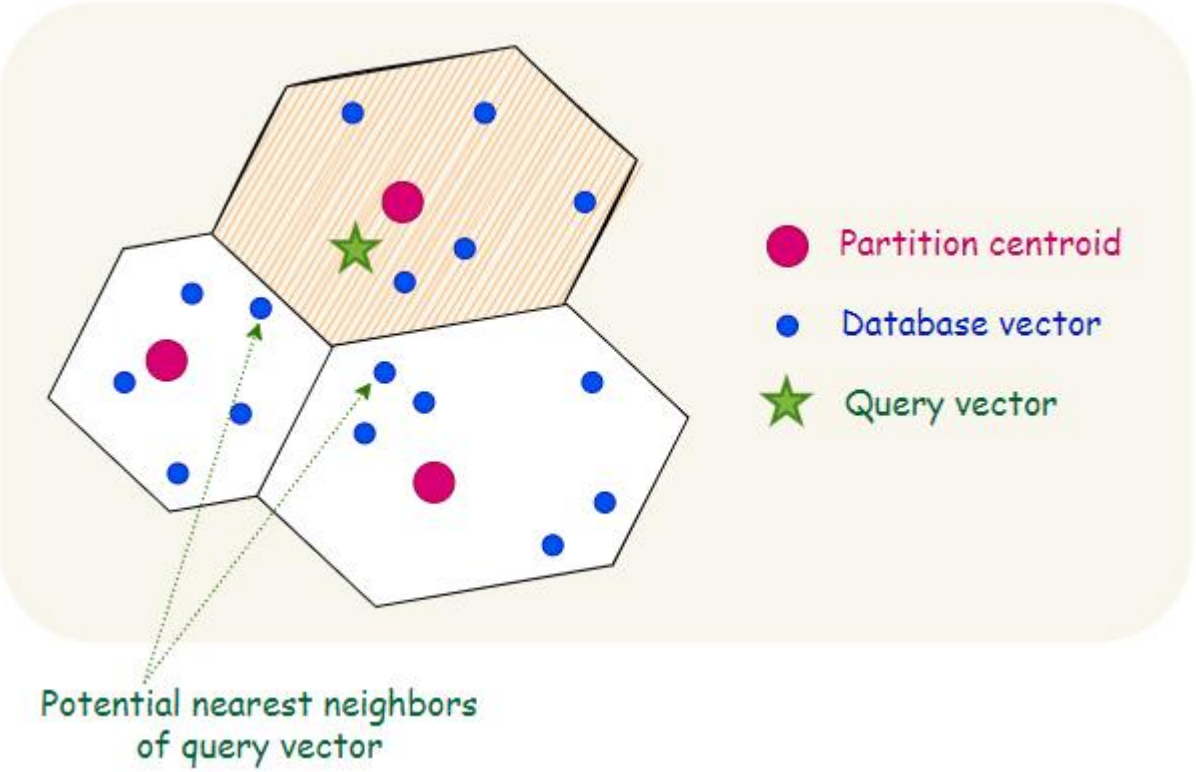
Dense Retrieval Efficiency

- With dense vectors, can't use inverted index/posting list trick
- Cell-probe/Inverted File Index (IVF)
  - Partition your vector space into K "cells"
    - E.g., run K-means clustering on the document vectors
  - Store list mapping each cell ID to corresponding list of documents
    - Similar to posting lists
    - Also store *residual vector* = difference between original & centroid



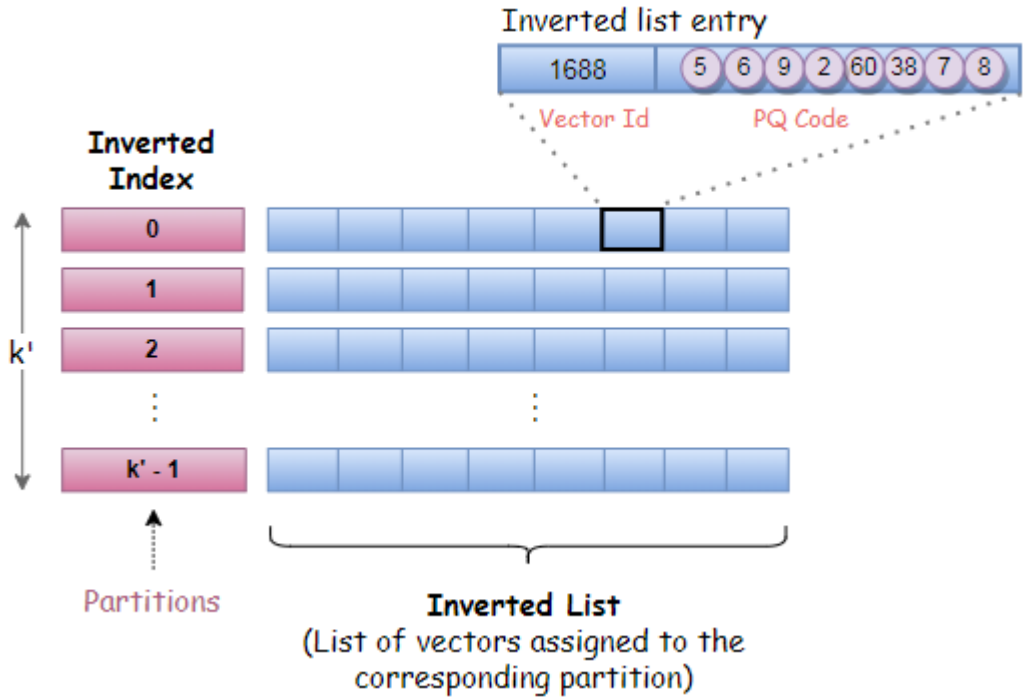
Using Cell Probe/IVF

- Compute query vector
- Identify nearest centroid(s)
- Compute full score for documents that mapped to same centroid
  - Use residual vector to help compute the exact score



Product Quantization (PQ)

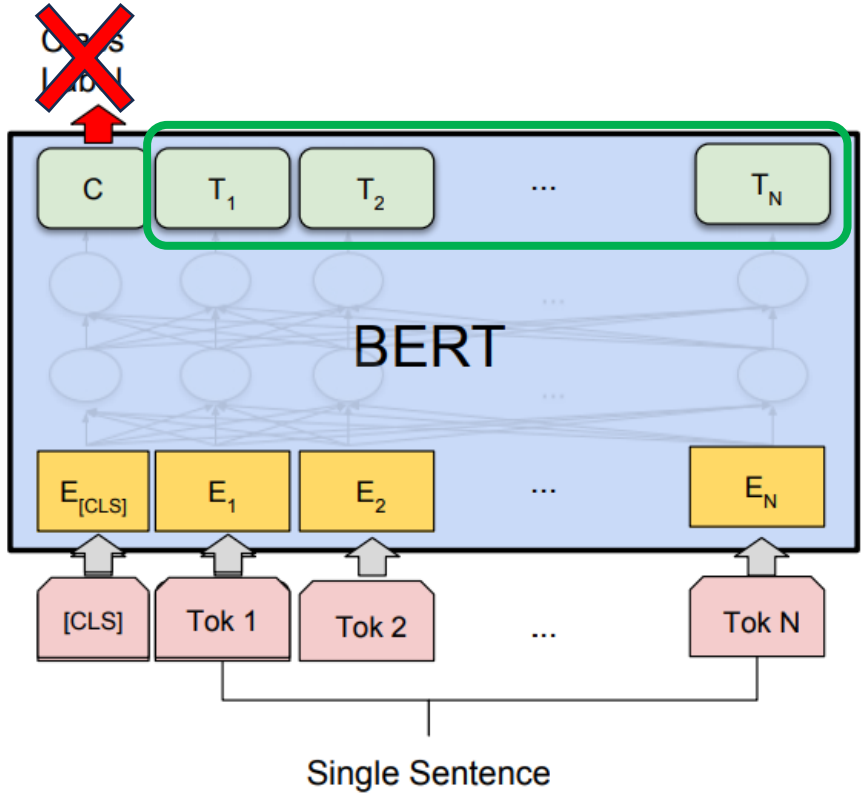
- What about memory?
  - BERT had 768-dimensional vectors x 4-byte floats x 21M chunks = 64.5 GB
  - This is pretty expensive!
- Product Quantization idea
  - Break residual vector into smaller vectors
  - Cluster these small vectors & map them to centroids
  - Example: If each small vector is 16-dimensions & we make 65536 clusters, we only need 2 bytes per 16 dimensions!
  - 32x savings compared to 4 byte floats
- Combined index called IVFPQ



Single Dense Vector Summary

- **Good**: Can leverage pre-trained neural models to generate meaningful vector representations
  - Methods after DPR use similar techniques with newer LMs
- **Good**: IVFPQ techniques enable efficient querying
  - But not quite as fast/memory efficient as sparse vectors
- **OK but not ideal**: Expensive setup cost
  - Have to train encoder, gather data and create negative passages
  - Have to run encoder on entire document collection—expensive to add/change a lot of documents
- **OK but not ideal**: Single vector struggles to capture everything about a document/passage

Beyond a Single Vector

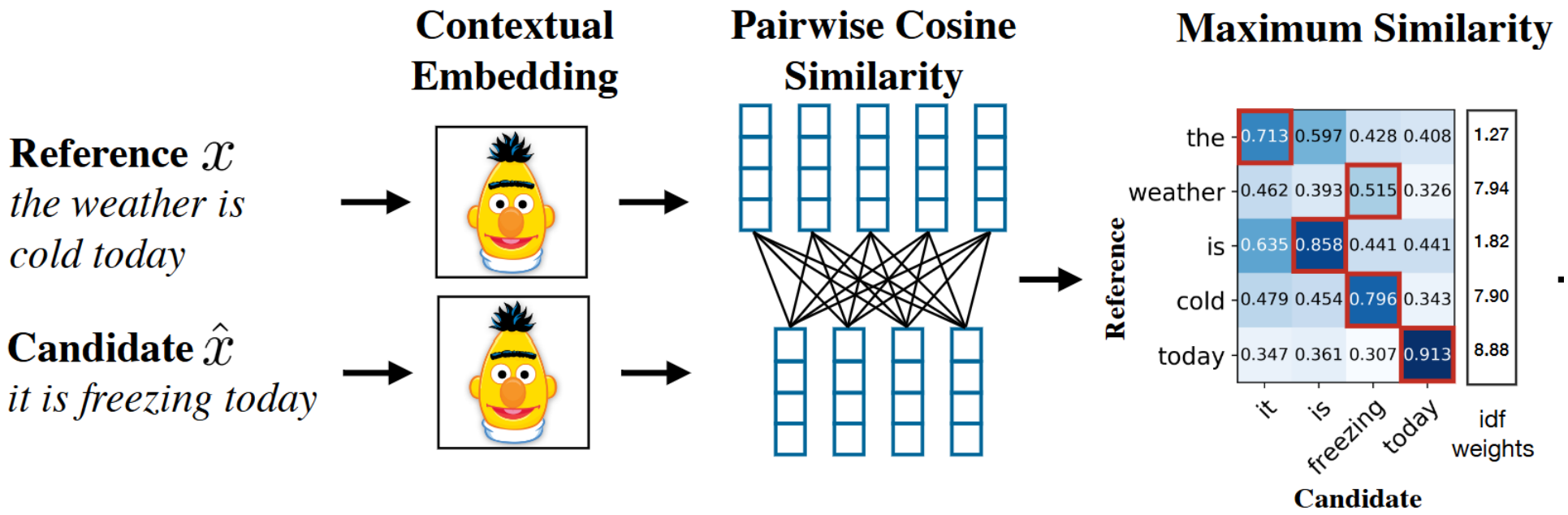


BERT generates a **separate vector representation for every token**—can we use them?

"You can't cram the meaning of a whole %&!$# sentence into a single $&!#* vector!"

Prof. Ray Mooney

Recall: BERTScore



- Use BERT to embed reference & predicted sentences
- For each reference word, identify most similar predicted word based on BERT embeddings + cosine similarity
- Average similarity of these pairs is BERTScore Recall
- Can do inverse to get BERTScore Precision, Harmonic mean = BERTScore F1

ColBERT Scoring

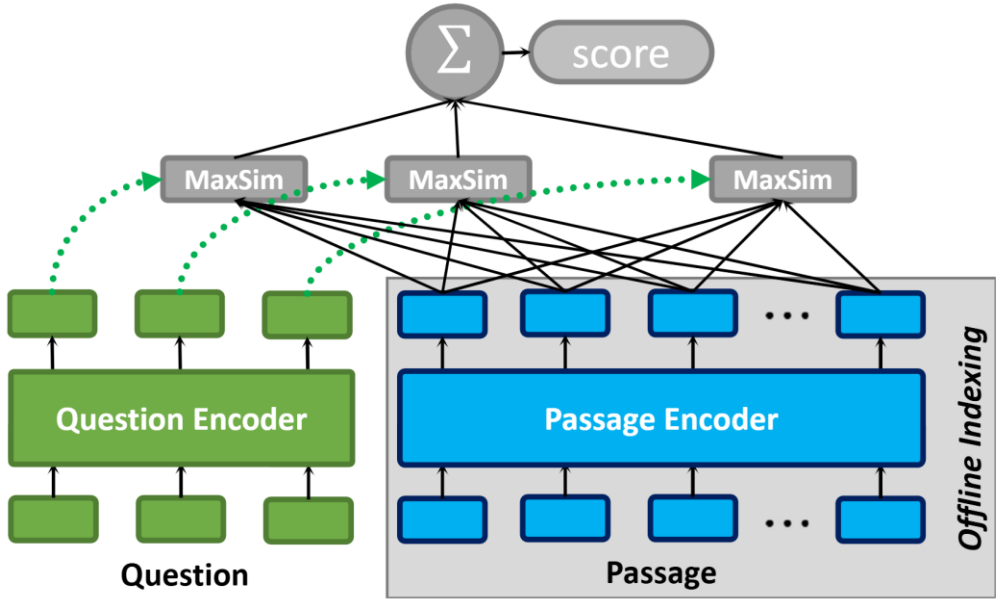
- Run BERT encoder on query to obtain vectors $q_1, \ldots, q_n$
- Run BERT encoder on passage to obtain vectors $p_1, \ldots, p_n$
- Score:

$$\mathrm{sim}(q,p) = \sum_{i=1}^{n} \max_{j=1}^{m} q_i^\top p_j$$

For each query word…

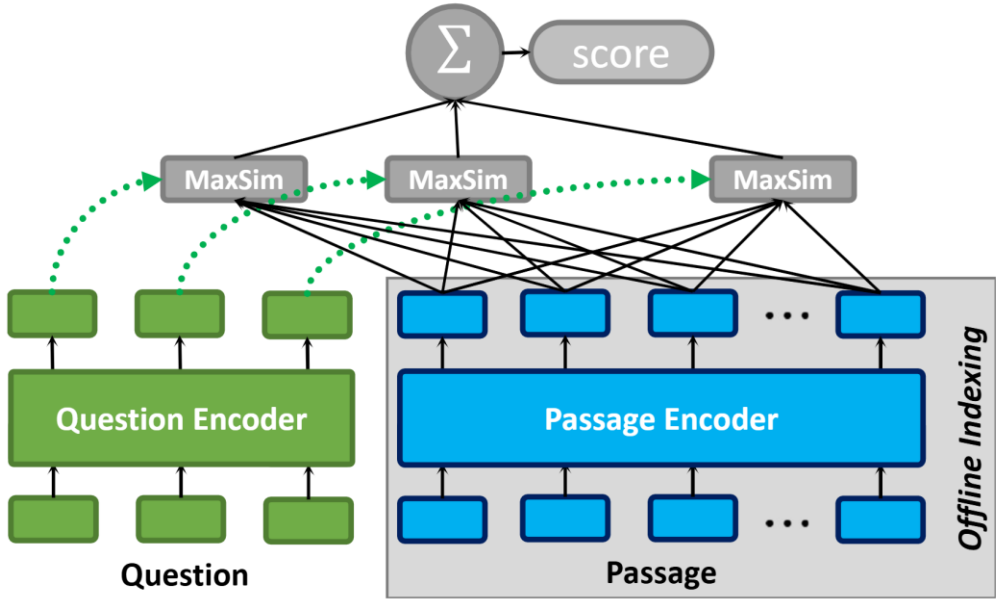
…find best matching passage word

- Equivalent to BERTScore
- Training: Similar to DPR, just with a different definition of similarity



ColBERT Intuition

- Encode question & passage separately, compare the representations at the end
  - A “Late Interaction” approach
  - Very similar to “Late Fusion” of CLIP
- Compare with an “Early Fusion” BERT approach: Input question & passage to model together
  - Many layers of cross-attention enable deep processing of query-passage relationship
  - But can’t retrieve efficiently



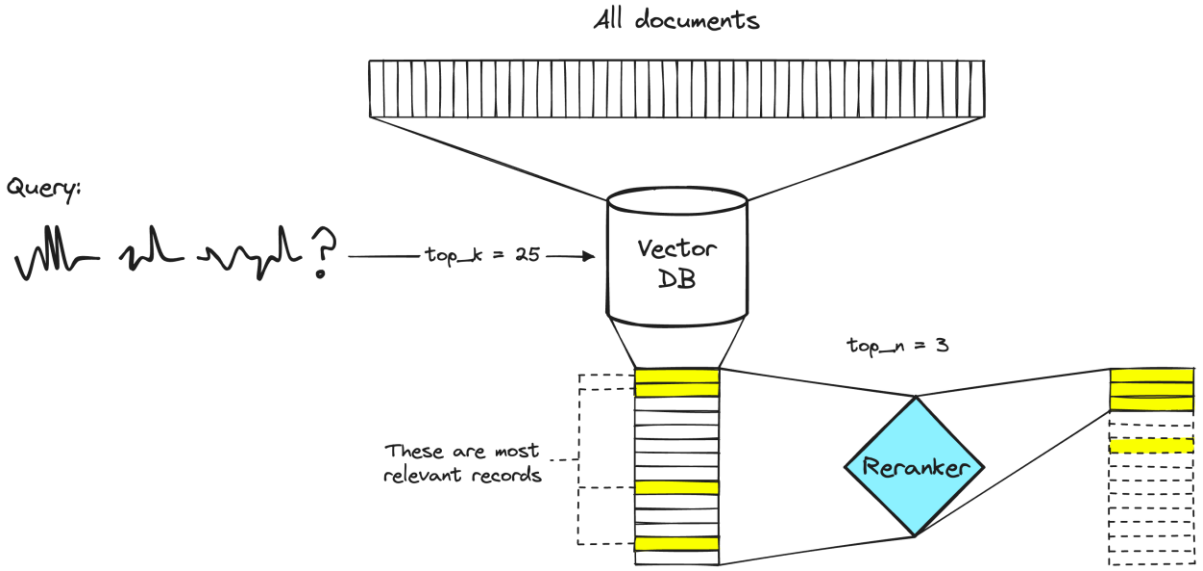
ColBERT Retrieval

$$\text{sim}(q,p) = \sum_{i=1}^{n} \max_{j=1}^{m} q_i^\top p_j$$

- Pre-compute & store **vectors for each token in every document** in a big IVFPQ index
  - Much more memory than a single vector index!
- Given query, compute query vectors q_1, …, q_n
- For each query vector, map to centroid, identify closest documents
  - This lower-bounds $\max_{j=1}^{m} q_i^\top p_j$ for one index *i* & each nearby passage *p*
- Add up the lower bounds over all query words for each passage ID
  - Quick estimate of the full ColBERT score
- Keep candidates with best lower bounds, score them fully & rank

Re-ranking

- Any expensive scoring model can be used to “re-rank” a cheaper retrieval method
- Typical: Use sparse method for initial retrieval (fastest)
- Option 1: Re-rank using full BERT model
  - Feed (query, document) as single sequence to BERT
- Option 2: Re-rank using ColBERT
  - Advantage: Passage vectors are pre-computed, only need to run BERT on query at inference time
  - Disadvantage: “Late Interaction” only

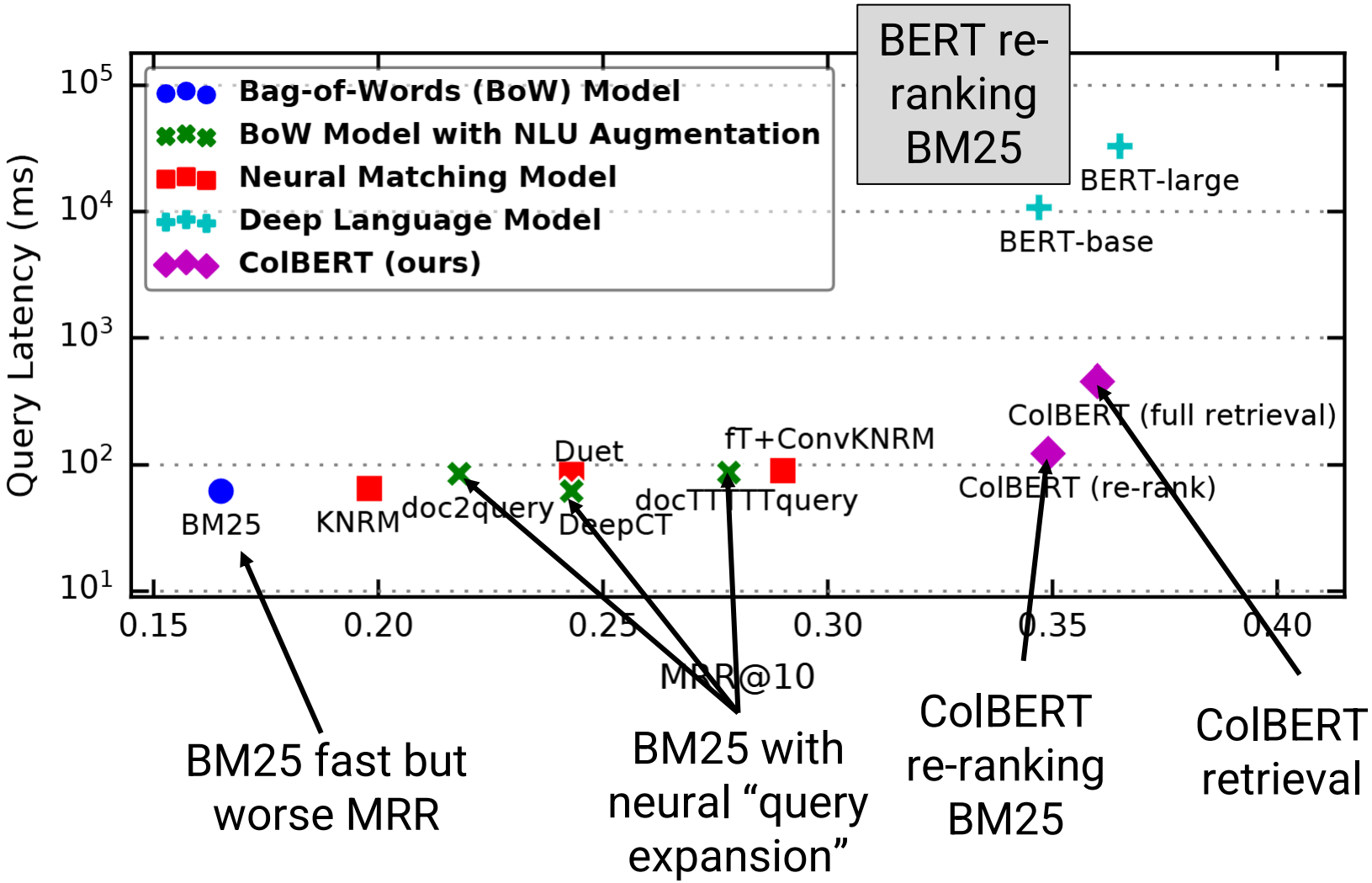


Evaluating IR

- Precision & Recall are still valid metrics
  - Test example = query & some documents known to be relevant
  - **Precision@K**: Of the top K retrieved documents, how many were relevant?
  - **Recall@K**: How many relevant documents are in the top K retrievals?
- **Mean Reciprocal Rank (MRR)** is also common
  - Find the rank of the highest-ranked relevant document
  - Compute 1/rank & average across test set
  - In English: You get…
    - 1 point if first good answer is ranked #1
    - 1/2 points if first good answer is ranked #2
    - And so on…

Performance Comparison

- Figure 1 of the ColBERT paper
- BM25: Fastest but low MRR (on this dataset)
- Query expansion helps
- ColBERT full retrieval high MRR but slower
- ColBERT re-ranking BM25 slightly worse, but faster
- ColBERT much faster than full BERT re-ranking BM25, similar MRR



ColBERT summary

- **Good**: Represents queries and passages with 1 vector per token, enabling finer-grained meaning representation
- **OK but not ideal**: Retrieval is sped up by IVFPQ tricks, but still slower than other methods
- **OK but not ideal**: Still has expensive setup cost

Outline

- Information Retrieval
  - Problem Definition
  - Sparse Retrieval
  - Dense Retrieval & Neural Models
- **Retrieval Augmented Generation**

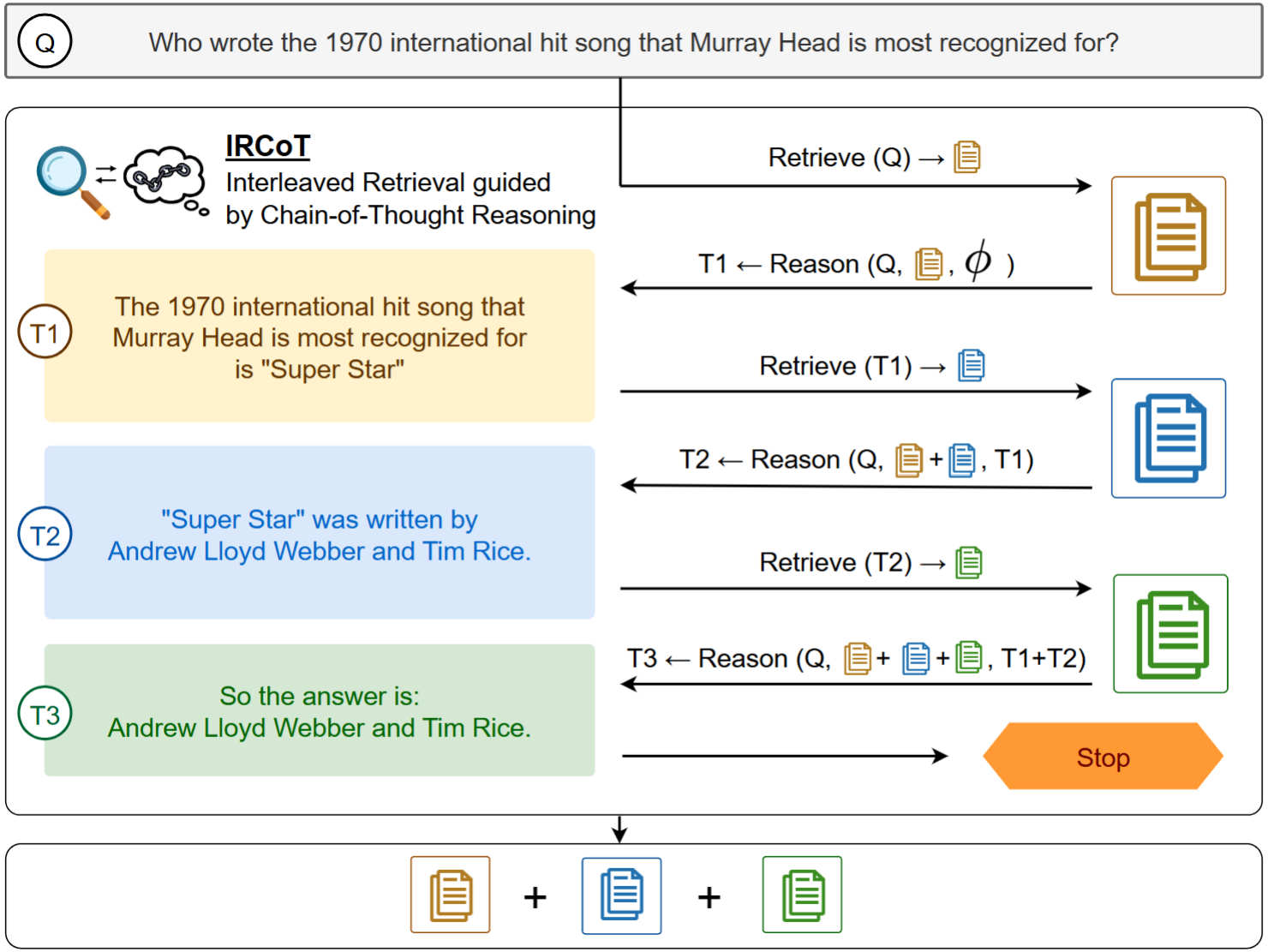
Retrieval Augmented Generation (RAG)

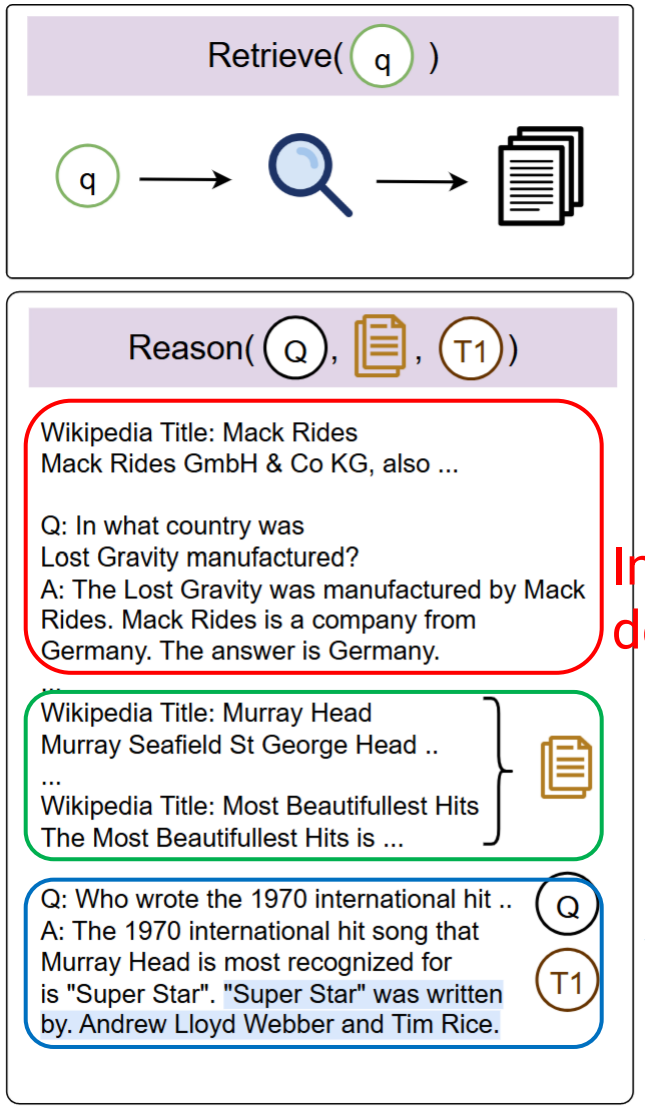
- Consider some question posed to an LLM:
  - *Who won the first Nobel Prize in physics?*
  - *How can I take a 500 level class as an undergraduate?*
- First retrieve top K relevant passages from appropriate database
  - First query: Wikipedia or other general knowledge source
  - Second query: Internal university handbook/documentation
- Provide question + documents as context to LLM
  - LLM generates a response

Multi-stage RAG

- Query: *In what city was Facebook founded?*
- Suppose retriever returns things like:
  - *Facebook was founded by Mark Zuckerberg while at Harvard University*
- This is useful! Now you just need to retrieve where Harvard is
- This motivates doing **multiple stages of RAG**
  - Interleave retrieval and LLM-driven "reasoning"
  - If enough information, output answer and stop
  - If not enough information, formulate a new query to gather more information

Example of Multi-stage RAG: IRCoT





Module #1: Standard retrieval

Module #2: LLM reasoning

In-context demonstration

Retrieved passages

Attempt to reason about answer

OpenAI Deep Research

User query & LM output

I am a researcher interested in bringing mixed-gas sorption capabilities to my lab. Please discuss the differences between pure- and mixed-gas sorption for glassy polymers, how the dual-mode sorption model can be used to predict mixed-gas sorption behavior in glassy polymers (include equations where applicable), and what challenges there are in accurately predicting pure- and mixed-gas sorption using the dual-mode sorption model. Only include open-source information as of October 1, 2024.

Pure vs. mixed-gas sorption in glassy polymers: In a glassy polymer (below T_g), a single gas typically exhibits a concave sorption isotherm: uptake rises sublinearly with pressure, unlike the linear Henry's-law behavior above T_g. This is attributed to

Internal LM Reasoning

Understanding sorption models

I'm gathering info on pure and mixed-gas sorption in glassy polymers, focusing on the dual-mode sorption model and addressing challenges in modeling predictions.

[search action]

Assessing open access

OK, let me see. The result from MDPI on mixed-gas sorption in glassy polymers is probably open access.

- LLM interleaves writing in natural language with calling "search action"

Conclusion

- Information retrieval: Simple methods are still commonly used for efficiency boost
- Neural methods can be used as retrievers or re-rankers
- Retrieval enables LLMs to handle knowledge-intensive queries